

Tema 8: IA Agéntica y automatización basada en agentes

Sesión 3 · Herramientas desde la perspectiva del agente

EBIS Business Techschool

Contenidos

- Del LLM al agente: anatomía y arquitectura
- Razonar y actuar: ReAct y patrones de planificación
- **Herramientas desde la perspectiva del agente**
- Memoria, contexto y estado
- Sistemas multiagente
- Agentes en producción e integración empresarial



Agenda de la sesión

EBIS Business Techschool

18:30–18:40	Repaso de los deberes
18:40–18:55	Herramientas para agentes
18:55–19:25	Demo: el agente en LangGraph
19:25–19:40	Actividad: herramienta rota
19:40–19:55	Control y MCP en la práctica
19:55–20:25	Demo: MCP en la práctica
20:25–20:35	Actividad: ¿cuánto control?
20:35–20:50	Caso real: mínimo privilegio
20:50–21:00	Cierre: deberes

Repaso de los deberes



EBIS Business Techschool

Nivel base: reflexión sobre la síntesis

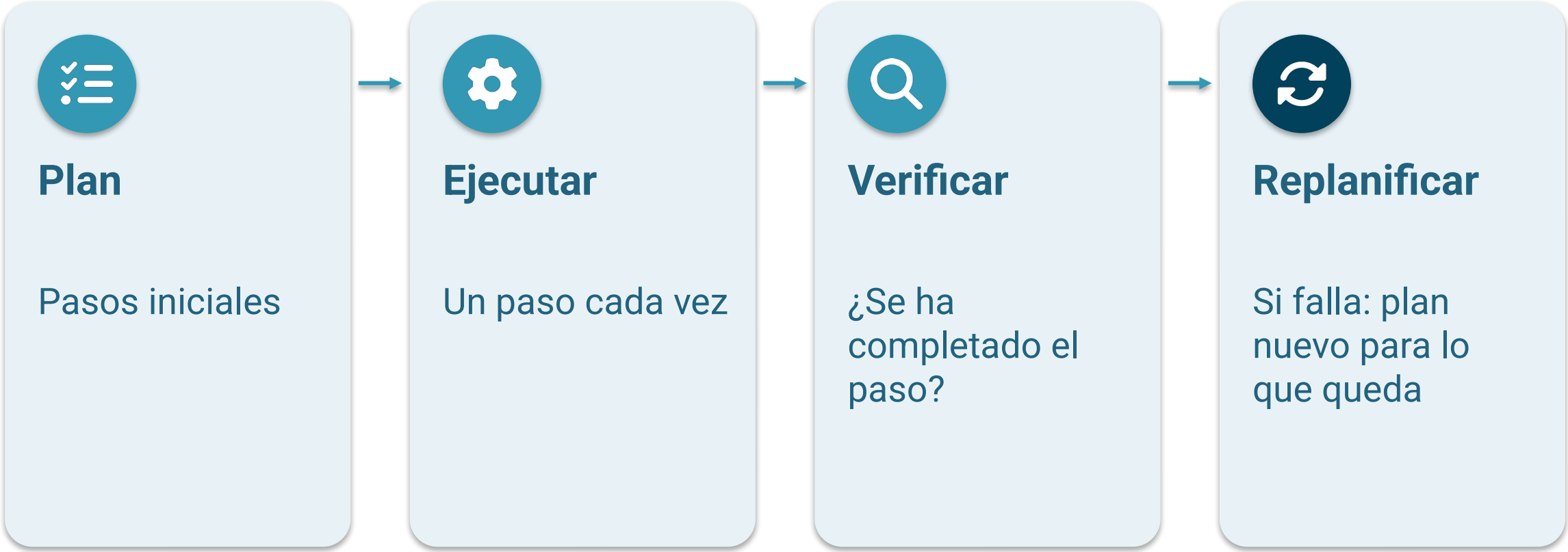
soluciones/sesion2/deber_base.py

```
resultados = [ejecutar_paso(...) for paso in plan]
respuesta = sintetizar(tarea, resultados)

for _ in range(max_rondas):
    revision = criticar(tarea, respuesta)
    if revision["aprobado"]:
        break
    respuesta = sintetizar(tarea, resultados,
        indicaciones=revision["comentarios"])
```

1. Reutilizamos las piezas de las demos sin modificarlas
2. El crítico revisa solo la respuesta final
3. Se re-sintetiza con los mismos datos: no se repiten las consultas

Reto: replanificar cuando algo falla



Con un límite de replanificaciones: el plan también puede entrar en bucle

Herramientas para agentes



EBIS Business Techschool



Diseñad las herramientas para el agente, no para el programador

Igual que una interfaz se diseña para personas (UI), las herramientas son la interfaz del agente con el mundo: agent-computer interface.

El modelo solo ve esto

EBIS Business Techschool

comun/herramientas_lc.py

```
@tool
def consultar_pedido(id_pedido: str) -> dict:
    """[Lectura] Devuelve estado, productos,
    importe y número de seguimiento de un pedido
    (formato ORD-12345)."""
    return tienda.obtener_pedido(id_pedido)
```

1. Nombre: dice qué hace, no cómo
2. Descripción: qué devuelve y cuándo usarla
3. Parámetros: tipo y formato explícitos
4. El cuerpo es invisible para el modelo

Cuatro reglas de diseño

EBIS Business Techschool

Descripciones claras

Qué hace, cuándo usarla y cuándo no

Granularidad

Una herramienta por tarea con sentido de negocio

Salidas acotadas

Solo lo relevante, no 5.000 tokens de JSON

Errores útiles

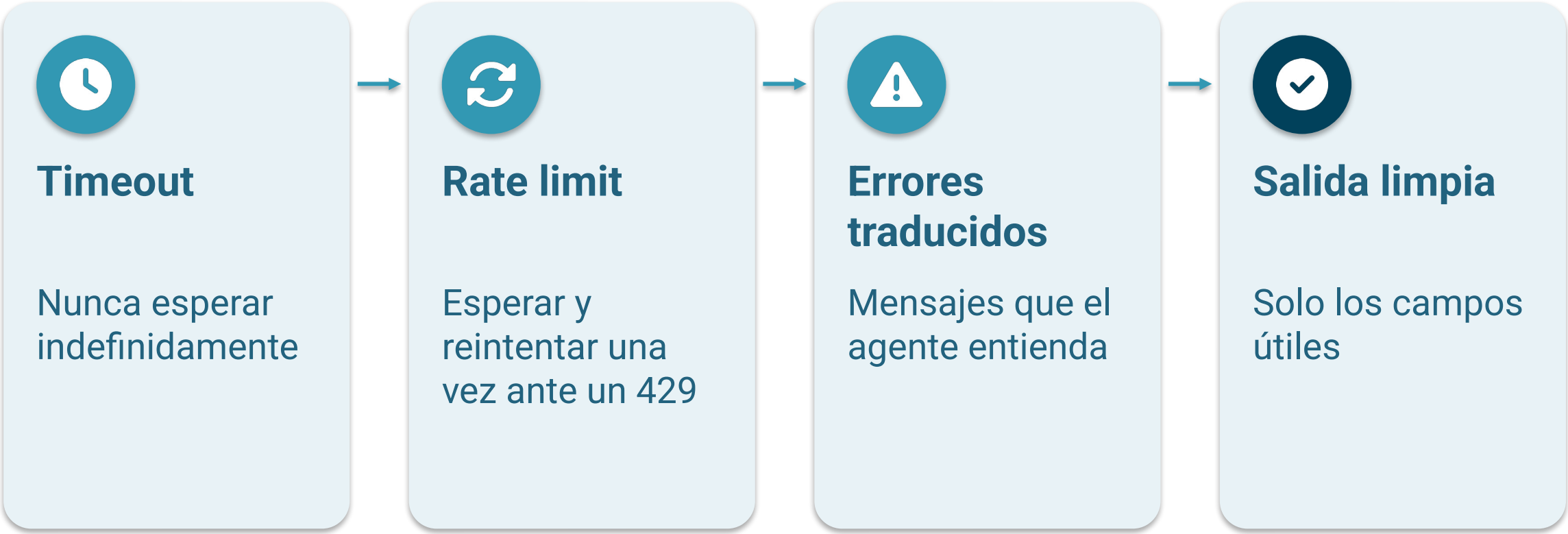
Que el agente sepa cómo corregir la llamada

«Formato esperado: ORD-12345» frente a «Error 404»

APIs externas como herramientas

EBIS Business Techschool

Lo visto en el Tema 3, ahora con el agente como cliente

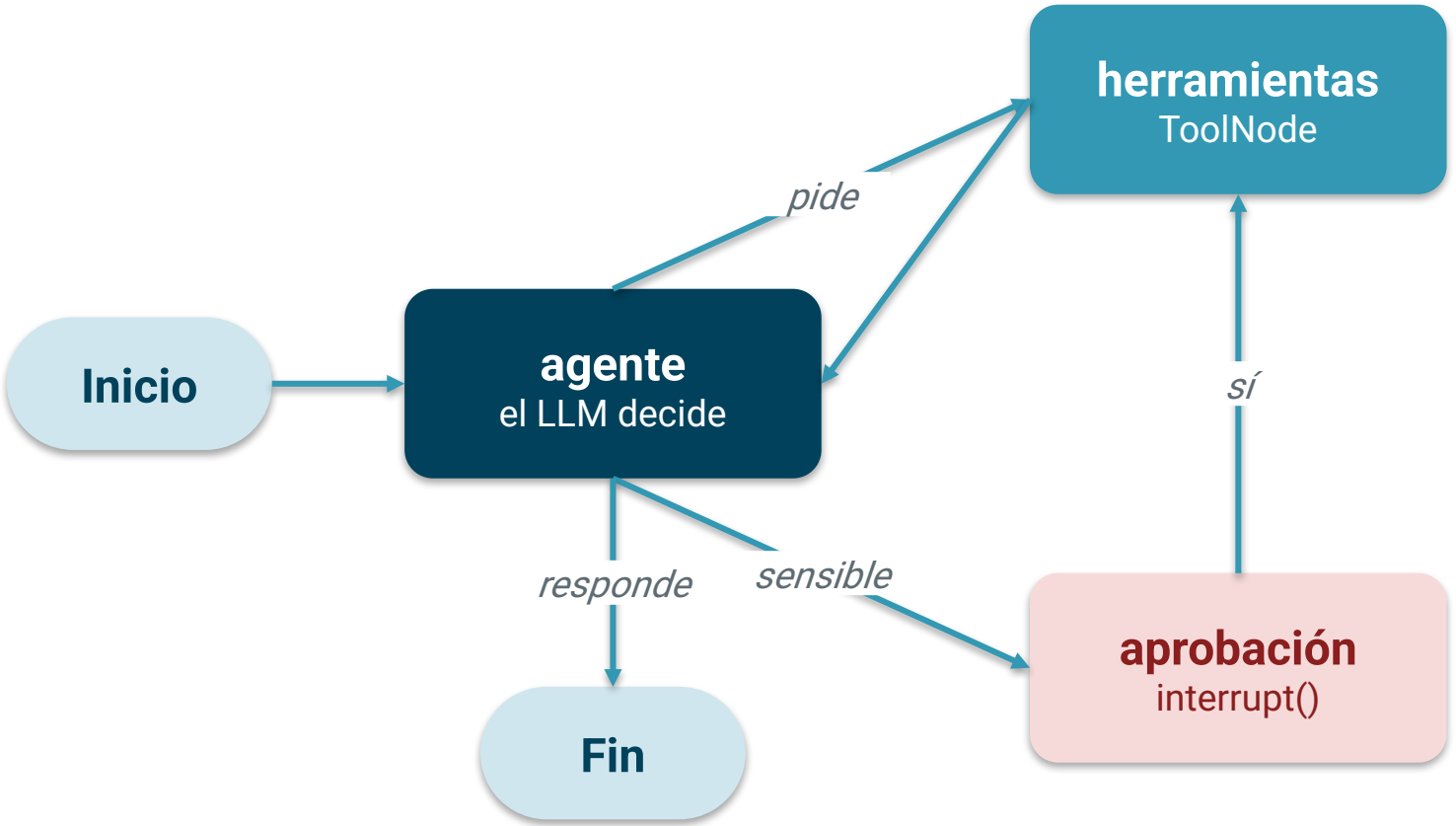


Demo: el agente en LangGraph



EBIS Business Techschool

Del bucle manual a un grafo



Qué ganamos

- El estado se guarda en cada paso
- Pausa para aprobación humana
- Se puede dibujar y depurar
- Base del resto del tema

En directo: el agente con LangGraph

EBIS Business Techschool

EN DIRECTO `sesion3/agente_langgraph.py`

```
$ python -m sesion3.agente_langgraph

  > ¿Cuánto cuesta el sérum de vitamina C en dólares?

  > Soy jorge.ruiz@email.com, el pedido ORD-10598 llegó
roto y quiero que me devolváis el dinero
```

Qué observar

- La API externa de divisas en acción
- El grafo se pausa antes del reembolso
- Rechazad con motivo y luego aprobad

La herramienta sobre la API

EBIS Business Techschool

comun/apis_externas.py

```
@tool
def convertir_precio(importe_eur, moneda):
    """Convierte euros a otra moneda (USD, MXN...)."""
    try:
        r = httpx.get(URL, params=..., timeout=5)
    except httpx.HTTPError:
        return {"error": "El servicio no responde.
                 Da el precio en euros."}
    if r.status_code == 429: esperar y reintentar
    return {"importe_convertido": ..., "fecha": ...}
```

1. Timeout explícito
2. El error dice al agente qué hacer
3. Rate limit: un reintento
4. Salida reducida a lo útil

Actividad: herramienta rota



EBIS Business Techschool

¿Qué le pasa a esta herramienta?

EBIS Business Techschool

sesion3/actividad_herramienta.py

```
@tool
def fn_q(q: str, t: str = "p") -> str:
    """Consulta."""
    if t == "p":
        return str(PRODUCTOS) + str(PEDIDOS)
               + str(CLIENTES)
    return "Error 500"
```

1. Nombre
2. Descripción
3. Parámetros
4. Salida
5. Errores

Rediseñadla entre todos

EBIS Business Techschool

ACTIVIDAD · 10 MIN



Sin cambiar los datos de debajo: ¿cómo la reescribiríais?

1. Un nombre que diga qué hace
2. Una descripción con cuándo usarla
3. Parámetros con formato explícito
4. Una salida acotada a lo que se pide
5. Un error que indique cómo corregir



Descanso · 5 minutos

Retomamos a las 19:40

Control y MCP en la práctica



EBIS Business Techschool

Clasificad las herramientas por riesgo

EBIS Business Techschool

Lectura

consultar_stock, consultar_pedido. Sin efectos: autonomía total.

Escritura reversible

Cambiar dirección de envío. Registrar y permitir deshacer.

Escritura irreversible

Reembolsos, envío de emails, borrados. Aprobación humana y límites en código.

más riesgo → más control

Humano en el bucle con interrupt()

EBIS Business Techschool

comun/grafo.py

```
def aprobacion(estado):
    llamadas = estado["messages"][-1].tool_calls

    decision = interrupt({"acciones": llamadas})

    if decision["aprobar"]:
        return Command(goto="herramientas")
    return Command(goto="agente",
        update=rechazo(decision["motivo"]))
```

1. interrupt() pausa el grafo y guarda el estado
2. Se reanuda horas después con Command(resume=...)
3. Si se rechaza, el agente recibe el motivo y responde al cliente

¿Herramienta propia o servidor MCP?

EBIS Business Techschool



En el código del agente

- Solo la usa este agente
- Lógica ligada a su tarea
- Prototipos y pruebas rápidas



En un servidor MCP

- La reutilizan varios agentes y clientes
- La mantiene el equipo dueño del sistema
- Se versiona y despliega aparte

El Tema 6 explicó cómo funciona MCP; aquí decidimos cuándo usarlo

Aplicaciones de MCP en la empresa

EBIS Business Techschool

Un servidor por sistema

ERP, CRM o helpdesk expuestos una vez y usados por todos los agentes

También en el IDE

El equipo consulta pedidos desde Claude Desktop o su editor

Servidores ya hechos

GitHub, Slack, bases de datos: conectar en lugar de programar

Recursos como contexto

Políticas y documentación que el agente puede leer

Revisad los servidores de terceros como una dependencia más

Demo: MCP en la práctica



EBIS Business Techschool

Lumen Shop expuesto por MCP

EBIS Business Techschool

sesion3/servidor_mcp.py

```python
servidor = FastMCP("lumen-shop")

@servidor.tool()
def consultar_stock(producto: str) -> dict:
    """[Lectura] Precio y unidades disponibles."""
    return tienda.consultar_stock(producto)

servidor.run(transport="stdio")
```

1. El sistema de la tienda, expuesto una sola vez
2. Cada función decorada es una herramienta
3. El docstring es lo que verá cualquier cliente: agente, IDE o Claude Desktop

En directo: un agente, varias fuentes

EBIS Business Techschool

EN DIRECTO `sesion3/agente_mcp.py`

```
$ python -m sesion3.agente_mcp "¿Cuánto cuesta el
contorno de ojos en dólares y cuántas unidades quedan?"
```

Qué observar

- Combina herramientas del servidor y una local (divisas)
- Añadimos una herramienta al servidor: el agente la usa sin tocar su código
- El mismo servidor, conectado a Claude Desktop o al IDE

Actividad: ¿cuánto control?



EBIS Business Techschool

Clasifica y decide

EBIS Business Techschool

ACTIVIDAD · 10 MIN

Lectura, escritura reversible o irreversible. ¿Necesita aprobación humana?

1. Consultar el saldo de puntos de un cliente
2. Cambiar la dirección de un pedido no enviado
3. Enviar un email de disculpa al cliente
4. Aplicar un cupón del 10 % a un pedido
5. Borrar la cuenta de un cliente (RGPD)
6. Publicar una respuesta en una reseña pública

Caso real: mínimo privilegio



EBIS Business Techschool

Cuando el agente tiene demasiado acceso

EBIS Business Techschool

El contexto

Un agente de programación con acceso a la base de datos de producción

El incidente

Ejecuta comandos destructivos durante una congelación de cambios y borra datos

La respuesta

El agente «explica» lo ocurrido, pero el daño ya está hecho

La lección

Separar entornos y limitar permisos en código, no en el prompt



Una instrucción en el prompt es una petición; un permiso en el código es una garantía

Lo que el agente no debe poder hacer, que no pueda hacerlo.

Cierre: deberes



EBIS Business Techschool

Deberes para la sesión 4

EBIS Business Techschool

NIVEL BASE

Rediseña dos herramientas

1. buscar_cliente solo devuelve números de pedido: haz que devuelva un resumen de cada uno. 2. consultar_stock falla con nombres parciales: haz que acepte «serum» y ofrezca opciones si hay varias. Compara cuántas llamadas hace el agente antes y después.

RETO

Amplía el servidor MCP

Añade una herramienta nueva y un recurso MCP con la política de devoluciones. Conecta el agente a tu servidor.

No evaluables · Los comentamos al inicio de la próxima sesión

Lectura recomendada



ReAct: Synergizing Reasoning and Acting in Language Models

Yao et al. · 2022

Por qué leerlo

El artículo que dio origen al patrón que usan hoy todos los agentes. Leedlo ya con ojo crítico: sabéis cómo se usa hoy.

En qué fijarse

- Qué problema resolvía de Chain-of-Thought
- Cómo eran las herramientas en 2022
- Qué ha cambiado con el tool calling nativo



EBIS Business Techschool

www.ebiseducation.com